

ui.navigate 全面详细解析

`ui.navigate` 是 NiceGUI 框架（2.0.0 版本及以上新增核心功能）提供的导航工具集，专注于实现浏览器历史操作、URL 跳转、外部链接打开等导航相关功能，支持单页应用内部导航与跨页面 / 跨域跳转，核心优势是 API 简洁、与 NiceGUI 组件生态深度兼容。

一、核心功能概览

`ui.navigate` 围绕“浏览器导航”和“URL 管理”两大场景，提供三类核心能力：

- 浏览器历史操作（前进、后退、刷新）；
- 页面 / URL 跳转（打开内部页面、外部链接，支持新标签页）；
- 浏览器历史 API 扩展（推送新 URL、替换当前 URL，不刷新页面）。

二、详细功能说明与使用示例

1. 浏览器历史基础操作

提供与浏览器原生历史功能对应的快捷方法，无需手动操作 DOM，直接通过 `ui.navigate` 调用：

方法名	功能描述	适用场景
<code>ui.navigate.back()</code>	回到浏览器历史的上一页	模拟浏览器“后退”按钮功能
<code>ui.navigate.forward()</code>	前进到浏览器历史的下一页	模拟浏览器“前进”按钮功能
<code>ui.navigate.reload()</code>	刷新当前页面	重新加载页面资源

示例代码（基础历史操作）

```
from nicegui import ui

# 用行布局包裹三个操作按钮
with ui.row():
    ui.button('Back', on_click=ui.navigate.back)    # 后退
    ui.button('Forward', on_click=ui.navigate.forward) # 前进
    ui.button('Reload', on_click=ui.navigate.reload) # 刷新

ui.run()
```

2. 页面 / URL 跳转 (`ui.navigate.to`)

`ui.navigate.to` 是核心跳转方法（替代旧版本 `ui.open`），支持多种跳转目标，灵活控制打开方式：

关键参数

参数名	类型	说明
target	页面函数 / 同页 NiceGUI 元素 / 字符串	跳转目标: - 字符串: 绝对 URL (如 <code>https://github.com</code>) 或相对路径 (基于项目基准 URL) ; - 页面函数: NiceGUI 单页应用的内部页面 (如 <code>def page1(): ui.label('页面1')</code>) ; - 同页元素: 通过元素 ID 或引用跳转到页面内指定位置
new_tab	布尔值 (默认 <code>False</code>)	是否在新标签页打开目标; 注意 : 浏览器可能拦截非用户主动触发的新标签页 (如异步回调中打开), 此为浏览器安全策略, 无法通过应用修改

适用场景与示例

场景 1: 打开外部 URL (支持新标签页)

```
from nicegui import ui

url = 'https://github.com/zauberzeug/nicegui/'
# 点击按钮在新标签页打开 GitHub 仓库
ui.button('Open GitHub', on_click=lambda: ui.navigate.to(url, new_tab=True))

ui.run()
```

场景 2: 跳转到应用内部页面 (单页应用)

```
from nicegui import ui

# 定义内部页面函数
def home_page():
    ui.label('首页')
    ui.button('跳转到详情页', on_click=lambda: ui.navigate.to(detail_page))

def detail_page():
    ui.label('详情页')
    ui.button('返回首页', on_click=lambda: ui.navigate.to(home_page))

# 初始加载首页
ui.navigate.to(home_page)
ui.run()
```

场景 3：跳转到同页指定元素（锚点跳转）

```
from nicegui import ui

ui.button('跳转到底部', on_click=lambda: ui.navigate.to/footer)) # 直接引用元素

# 中间内容（占位）
ui.label('页面内容...' * 50)

# 页面底部元素（作为跳转目标）
footer = ui.label('页面底部')

ui.run()
```

注意事项

- 若需确保新标签页一定打开，优先使用 `ui.link` 组件（其 `new_tab` 参数兼容性更强），而非 `ui.navigate.to`；
- 相对路径跳转基于项目的 `base_url` 配置（默认 `/`），例如 `ui.navigate.to('/about')` 会跳转到 `http://localhost:8080/about`。

3. 浏览器历史 API 操作（`ui.navigate.history`）

2.13.0 版本新增，基于 JavaScript 的 `History API` 封装，支持在不刷新页面的前提下修改浏览器地址栏 URL，适用于单页应用（SPA）的路由管理：

方法名	功能描述	区别于普通跳转的核心优势
<code>ui.navigate.history.push(url)</code>	向浏览器历史栈添加一个新 URL，地址栏更新，页面不刷新	保留当前历史记录，可通过“后退”回到原 URL
<code>ui.navigate.history.replace(url)</code>	用新 URL 替换当前历史栈中的 URL，地址栏更新，页面不刷新，原 URL 被覆盖	不保留原 URL 历史，无法通过“后退”回到替换前的 URL

示例代码（历史 API 操作）

```
from nicegui import ui

# 推送新 URL 到历史栈（地址栏变为 http://localhost:8080/a，页面不刷新）
ui.button('Push URL', on_click=lambda: ui.navigate.history.push('/a'))

# 替换当前 URL（地址栏变为 http://localhost:8080/b，原 URL 被覆盖）
ui.button('Replace URL', on_click=lambda: ui.navigate.history.replace('/b'))

ui.run()
```

扩展说明

- 该功能依赖浏览器原生 `History API`，支持所有现代浏览器（Chrome、Firefox、Edge 等）；
- 适合用于单页应用的路由切换（如列表页→详情页），避免页面刷新导致的状态丢失；
- 若需配合路由逻辑（如根据 URL 加载对应内容），需自行监听地址栏变化（可结合 NiceGUI 的 `ui.route` 功能）。

三、版本兼容性与注意事项

1. 版本要求

- 基础功能（`back` / `forward` / `reload` / `to`）：NiceGUI $\geq 2.0.0$ ；
- 历史 API 功能（`history.push` / `history.replace`）：NiceGUI $\geq 2.13.0$ 。

2. 关键注意事项

- 新标签页拦截问题**：`ui.navigate.to(url, new_tab=True)` 可能被浏览器拦截，尤其是在异步操作（如定时器、网络请求回调）中调用时。解决方案：
 - 优先使用 `ui.link(url, '打开链接', new_tab=True)`，浏览器对 `<a>` 标签的新标签页打开策略更宽松；
 - 仅在用户主动点击按钮等交互事件中直接调用（避免异步嵌套）。
- 相对路径跳转**：`target` 为相对路径时，基于项目的 `base_url`（可通过 `ui.run(base_url='/my-app')` 配置），例如 `ui.navigate.to('about')` 会跳转到 `/my-app/about`。
- 历史 API 限制**：`history.push` / `history.replace` 仅修改地址栏 URL 和历史栈，不会自动加载页面内容，需手动配合逻辑（如根据 URL 渲染对应组件）。

四、核心总结

`ui.navigate` 是 NiceGUI 提供的“一站式导航解决方案”，核心价值在于：

- 统一 API：无需混用原生 JavaScript 或 DOM 操作，用 Python 语法即可完成所有导航需求；
- 场景覆盖全：从基础的前进后退，到复杂的 SPA 路由管理，均能支持；
- 生态兼容：与 NiceGUI 的组件（如 `ui.button` / `ui.link`）、页面函数无缝集成，开发体验一致。

根据需求选择合适的方法：

- 简单历史操作：用 `back` / `forward` / `reload`；
- 打开页面 / 链接：用 `navigate.to`（普通跳转）或 `ui.link`（新标签页优先）；
- SPA 路由管理：用 `history.push` / `history.replace`。

NiceGUI 中 ui.navigate 深度解析

在 NiceGUI 框架中，`ui.navigate` 是**页面导航的核心工具**，封装了前端路由的跳转逻辑，提供了声明式的页面跳转、历史记录管理、参数传递等能力。它与 `ui.page` 路由装饰器深度联动，是实现多页面应用中页面切换、导航控制的关键。`ui.navigate` 基于前端 Vue Router 实现，隐藏了底层的路由操作细节，让开发者仅通过 Python 代码即可完成复杂的导航逻辑。本文将从**核心定位**、**基础用法**、**参数传递**、**高级特性**、**导航守卫**、**常见问题**六个维度，对 `ui.navigate` 进行全方位的详细阐述。

一、ui.navigate 的核心定位

`ui.navigate` 是 NiceGUI 对前端路由跳转的 **Python 层封装**，其核心定位可总结为三点：

- 1. **页面跳转工具**：提供 `to()`、`back()`、`forward()` 等方法，实现页面的正向跳转、返回上一页、前进下一页等基础导航操作；
- 2. **路由状态管理**：与浏览器的历史记录深度联动，维护页面跳转的历史栈，支持基于历史记录的导航；
- 3. **参数传递载体**：支持在页面跳转时传递路径参数、查询参数，实现跨页面的数据传递；
- 4. **导航行为定制**：支持自定义跳转时的过渡动画、页面刷新策略，以及导航守卫（拦截未授权的跳转）。

`ui.navigate` 弥补了静态 `ui.link` 组件仅支持简单跳转的不足，实现了 **动态导航逻辑**（如登录后跳转、权限验证后跳转），是构建多页面应用的核心组件之一。

二、ui.navigate 的基础用法

`ui.navigate` 提供了一系列简洁的方法实现基础导航操作，核心方法包括 `to()`、`back()`、`forward()`、`refresh()`，覆盖了绝大多数日常导航场景。

2.1 核心导航方法

方法	功能描述	示例
<code>ui.navigate.to(path)</code>	跳转到指定路径的页面，支持绝对路径和相对路径	<code>ui.navigate.to('/dashboard')</code>
<code>ui.navigate.back()</code>	返回浏览器历史记录的上一页	<code>ui.navigate.back()</code>
<code>ui.navigate.forward()</code>	前进到浏览器历史记录的下一页	<code>ui.navigate.forward()</code>
<code>ui.navigate.refresh()</code>	刷新当前页面，重新执行页面处理函数	<code>ui.navigate.refresh()</code>

2.2 基础跳转示例

```
from nicegui import ui

# 首页
@ui.page('/')
def index():
    ui.label('首页').classes('text-3xl font-bold')
    # 跳转到关于页
    ui.button('前往关于页', on_click=lambda: ui.navigate.to('/about'))
    # 跳转到用户页（带路径参数）
    ui.button('前往用户123的页面', on_click=lambda: ui.navigate.to('/user/123'))

# 关于页
@ui.page('/about')
def about():
    ui.label('关于页').classes('text-3xl font-bold')
    # 返回上一页
    ui.button('返回首页', on_click=lambda: ui.navigate.back())
    # 前进到下一页（若有历史记录）
```

```

    ui.button('前进', on_click=lambda: ui.navigate.forward())
    # 刷新当前页面
    ui.button('刷新页面', on_click=lambda: ui.navigate.refresh())

# 带路径参数的用户页
@ui.page('/user/{user_id}')
def user_page(user_id: str):
    ui.label(f'用户 ID: {user_id}').classes('text-3xl font-bold')
    ui.button('返回首页', on_click=lambda: ui.navigate.to('/'))

if __name__ in {'__main__', '__mp_main__':
    ui.run(port=8080)

```

关键说明:

- `ui.navigate.to(path)` 的 `path` 参数支持**绝对路径** (以 `/` 开头), 这是 NiceGUI 中最常用的跳转方式;
- `ui.navigate.back()` / `forward()` 依赖浏览器的历史记录栈, 若没有对应的历史记录, 调用后不会产生任何效果;
- `ui.navigate.refresh()` 会重新执行当前页面的处理函数, 适用于需要更新页面数据的场景。

三、ui.navigate 的参数传递

在页面跳转时, `ui.navigate` 支持通过**路径参数**和**查询参数**两种方式传递数据, 分别适用于不同的业务场景, 与 `ui.page` 的参数接收逻辑完全兼容。

3.1 路径参数传递

路径参数是将数据嵌入到路由路径中, 适用于**资源标识** (如用户 ID、文章 ID) 等场景, 语法为在路径中使用 `{参数名}`, 跳转时直接拼接参数值。

3.1.1 单个路径参数

```

from nicegui import ui

# 跳转时传递用户 ID
@ui.page('/')
def index():
    ui.button('前往用户456的页面', on_click=lambda: ui.navigate.to('/user/456'))

# 接收路径参数
@ui.page('/user/{user_id}')
def user_page(user_id: str):
    ui.label(f'接收到的用户 ID: {user_id}').classes('text-2xl')

if __name__ in {'__main__', '__mp_main__':
    ui.run()

```

3.1.2 多个路径参数

支持在路径中传递多个参数，页面处理函数需按顺序接收：

```
from nicegui import ui

# 传递用户 ID 和文章 ID
@ui.page('/')
def index():
    ui.button('用户123的文章789', on_click=lambda: ui.navigate.to('/user/123/article/789'))

# 接收多个路径参数
@ui.page('/user/{user_id:int}/article/{article_id:str}')
def article_page(user_id: int, article_id: str):
    ui.label(f'用户 ID: {user_id} (整数)').classes('text-xl')
    ui.label(f'文章 ID: {article_id} (字符串)').classes('text-xl')

if __name__ in {'__main__', '__mp_main__':
    ui.run()
```

3.2 查询参数传递

查询参数是在 URL 后通过 `?key=value` 传递的参数，适用于**筛选条件**、**分页**、**搜索关键词**等场景，跳转时通过拼接字符串或字典传递，接收时通过 `ui.request.query` 获取。

3.2.1 拼接字符串传递查询参数

```
from nicegui import ui

@ui.page('/')
def index():
    # 传递搜索关键词和页码
    search_key = 'nicegui'
    page = 2
    ui.button('搜索结果页', on_click=lambda: ui.navigate.to(f'/search?q={search_key}&page={page}'))

# 接收查询参数
@ui.page('/search')
def search_page():
    q = ui.request.query.get('q', '默认关键词')
    page = ui.request.query.get('page', '1')
    ui.label(f'搜索关键词: {q}').classes('text-xl')
    ui.label(f'当前页码: {page}').classes('text-xl')

if __name__ in {'__main__', '__mp_main__':
    ui.run()
```

3.2.2 字典传递查询参数（进阶）

对于复杂的查询参数，可通过构建字典并转换为查询字符串传递（需使用 `urllib.parse`）：

```
from nicegui import ui
from urllib.parse import urlencode

@ui.page('/')
def index():
    # 构建查询参数字典
    query_params = {
        'q': 'python web',
        'page': 3,
        'sort': 'time'
    }
    # 转换为查询字符串
    query_str = urlencode(query_params)
    # 拼接路径并跳转
    ui.button('高级搜索', on_click=lambda: ui.navigate.to(f'/search?{query_str}'))

@ui.page('/search')
def search_page():
    # 遍历所有查询参数
    for key, value in ui.request.query.items():
        ui.label(f'{key}: {value}').classes('text-x1')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

3.3 路径参数与查询参数结合

适用于更复杂的场景，如“用户详情页 + 筛选该用户的文章类型”：

```
from nicegui import ui

@ui.page('/')
def index():
    # 同时传递路径参数（用户 ID）和查询参数（文章类型）
    ui.button('用户123的技术文章', on_click=lambda: ui.navigate.to('/user/123?type=tech'))

@ui.page('/user/{user_id:int}')
def user_page(user_id: int):
    # 接收路径参数和查询参数
    article_type = ui.request.query.get('type', 'all')
    ui.label(f'用户 ID: {user_id}').classes('text-2x1')
    ui.label(f'文章类型筛选: {article_type}').classes('text-x1')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```


四、ui.navigate 的高级特性

4.1 跳转时的页面刷新策略

`ui.navigate.to()` 支持通过 `replace` 参数控制是否替换浏览器的历史记录，以及是否强制刷新页面，核心参数：

- `replace: bool`：若为 `True`，则替换当前历史记录（而非添加新记录），调用 `navigate.back()` 时会跳过该页面；
- `force: bool`：若为 `True`，则强制刷新目标页面，即使路径未发生变化。

示例：替换历史记录

```
from nicegui import ui

@ui.page('/')
def index():
    ui.button('前往页面A（添加记录）', on_click=lambda: ui.navigate.to('/a'))
    ui.button('前往页面A（替换记录）', on_click=lambda: ui.navigate.to('/a', replace=True))

@ui.page('/a')
def page_a():
    ui.label('页面A').classes('text-3xl')
    ui.button('返回', on_click=lambda: ui.navigate.back())

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

说明：点击“替换记录”跳转后，再点击“返回”会直接回到首页（而非停留在页面 A 的上一条记录）。

4.2 基于会话的动态跳转

结合 `ui.session` 的会话状态，实现**条件跳转**（如登录后跳转到原目标页面、根据用户角色跳转不同页面）。

示例：登录后跳转回原页面

```
from nicegui import ui

# 鉴权装饰器：记录原目标页面
def require_login(page):
    def wrapper():
        if not ui.session.get('is_login'):
            # 记录原目标页面的路径
            ui.session['redirect_path'] = ui.request.url.path
            ui.notify('请先登录', type='error')
            return ui.navigate.to('/login')
        return page()
    return wrapper

# 登录页：跳转到记录的原页面
@ui.page('/login')
def login_page():
    def do_login():
```

```

        ui.session['is_login'] = True
        # 跳转到原目标页面，若无则跳转到首页
        redirect_path = ui.session.pop('redirect_path', '/')
        ui.navigate.to(redirect_path)

    ui.button('模拟登录', on_click=do_login)

# 受保护的页面
@ui.page('/dashboard')
@require_login
def dashboard():
    ui.label('仪表盘（需登录）').classes('text-3xl')

@ui.page('/profile')
@require_login
def profile():
    ui.label('个人中心（需登录）').classes('text-3xl')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

4.3 嵌套路由与子页面跳转

NiceGUI 支持通过 `ui.page` 实现嵌套路由，`ui.navigate` 可直接跳转到子页面路径，适用于复杂的页面结构（如后台管理系统的侧边栏导航）。

示例：嵌套路由跳转

```

from nicegui import ui

# 主页面（父路由）
@ui.page('/admin')
def admin():
    ui.label('后台管理系统').classes('text-3xl font-bold')
    # 跳转到子页面
    ui.button('用户管理', on_click=lambda: ui.navigate.to('/admin/users'))
    ui.button('角色管理', on_click=lambda: ui.navigate.to('/admin/roles'))

# 子页面1：用户管理
@ui.page('/admin/users')
def admin_users():
    ui.label('用户管理页面').classes('text-2xl')
    ui.button('返回后台首页', on_click=lambda: ui.navigate.to('/admin'))

# 子页面2：角色管理
@ui.page('/admin/roles')
def admin_roles():
    ui.label('角色管理页面').classes('text-2xl')
    ui.button('返回后台首页', on_click=lambda: ui.navigate.to('/admin'))

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()

```

4.4 跳转时的过渡动画

NiceGUI 基于 Quasar 的路由过渡动画, `ui.navigate` 支持通过配置页面的 `transition` 参数, 实现跳转时的动画效果 (如淡入淡出、滑动)。

示例: 带过渡动画的页面跳转

```
from nicegui import ui

# 首页: 配置淡入淡出动画
@ui.page('/', transition='fade')
def index():
    ui.label('首页 (淡入淡出)').classes('text-3xl')
    ui.button('前往关于页', on_click=lambda: ui.navigate.to('/about'))

# 关于页: 配置滑动动画
@ui.page('/about', transition='slide-right')
def about():
    ui.label('关于页 (从右滑动)').classes('text-3xl')
    ui.button('返回首页', on_click=lambda: ui.navigate.back())

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

支持的过渡动画: `fade`、`slide-left`、`slide-right`、`slide-up`、`slide-down`、`scale` 等 (参考 Quasar 路由过渡动画)。

五、ui.navigate 与导航守卫

导航守卫是指在页面跳转前执行的拦截逻辑, 用于实现权限验证、登录检查、数据预加载等功能。NiceGUI 中没有内置的导航守卫 API, 但可通过**装饰器**和**中间件**结合 `ui.navigate` 实现自定义的导航守卫。

5.1 页面级导航守卫 (装饰器)

通过装饰器在页面渲染前拦截跳转, 是最常用的导航守卫方式 (前文鉴权案例的核心逻辑):

```
from nicegui import ui

# 导航守卫: 检查管理员权限
def require_admin(page):
    def wrapper():
        # 检查会话中的用户角色
        if ui.session.get('role') != 'admin':
            ui.notify('无管理员权限', type='error')
            return ui.navigate.to('/') # 跳转到首页
        return page()
    return wrapper

@ui.page('/')
def index():
    ui.button('模拟管理员登录', on_click=lambda: ui.session.update({'role': 'admin'}))
    ui.button('前往管理员面板', on_click=lambda: ui.navigate.to('/admin'))
```

```
# 受保护的管理员面板
@ui.page('/admin')
@require_admin
def admin_panel():
    ui.label('管理员专属面板').classes('text-3xl text-red-500')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

5.2 全局导航守卫（中间件）

通过 FastAPI 中间件实现全局的导航拦截，适用于需要对所有页面跳转进行统一验证的场景：

```
from nicegui import ui, app
from fastapi import Request, HTTPException

# 全局导航守卫：拦截未登录用户访问受保护路径
@app.middleware('http')
async def auth_middleware(request: Request, call_next):
    # 排除公开路径
    public_paths = ['/', '/login']
    if request.url.path not in public_paths:
        # 检查会话中的登录状态
        session_id = request.cookies.get('nicegui-session')
        if not session_id or not app.storage.session.get(session_id, {}).get('is_login'):
            # 重定向到登录页
            raise HTTPException(status_code=307, headers={'Location': '/login'})
    response = await call_next(request)
    return response

@ui.page('/login')
def login_page():
    def do_login():
        app.storage.session[ui.session.id]['is_login'] = True
        ui.navigate.to('/dashboard')

    ui.button('模拟登录', on_click=do_login)

@ui.page('/dashboard')
def dashboard():
    ui.label('受保护的仪表盘').classes('text-3xl')

if __name__ in {'__main__', '__mp_main__'}:
    ui.run()
```

六、ui.navigate 的常见问题与解决方案

6.1 跳转后参数接收不到

问题：使用 `ui.navigate` 传递参数后，目标页面无法接收到参数。

解决方案：

1. 检查路径参数的命名是否与页面处理函数的参数名一致；
2. 查询参数需通过 `ui.request.query.get(key)` 获取，而非函数参数；
3. 确保路径参数的类型注解与传递的参数类型匹配（如 `{user_id:int}` 需传递整数）。

6.2 跳转后页面未刷新

问题：跳转到同一页面但参数不同时，页面数据未更新。

解决方案：

1. 使用 `ui.navigate.to(path, force=True)` 强制刷新页面；
2. 在页面处理函数中直接读取参数并更新组件，而非依赖缓存。

6.3 历史记录混乱

问题：多次跳转后，`navigate.back()` 无法回到预期的页面。

解决方案：

1. 使用 `ui.navigate.to(path, replace=True)` 替换历史记录，减少无效记录；
2. 避免频繁的嵌套跳转，简化路由结构。

6.4 跨页面状态丢失

问题：跳转后 `ui.session` 中的状态数据丢失。

解决方案：

1. 确认 `ui.session` 的使用在正确的上下文内；
2. 对于需要跨页面保留的状态，使用 `app.storage.user` 进行持久化；
3. 避免在页面跳转前清空 `ui.session`。

6.5 跳转时出现 404 错误

问题：调用 `ui.navigate.to(path)` 后，页面显示 404。

解决方案：

1. 检查路径是否与 `ui.page` 定义的路由路径一致；
2. 路径参数的类型不匹配时（如传递字符串给 `{user_id:int}`），会返回 404，需确保类型一致；
3. 确保动态注册的页面已通过 `ui.add_page()` 完成注册。

七、ui.navigate 与 ui.link 的对比

`ui.link` 是 NiceGUI 提供的静态导航组件，`ui.navigate` 是动态导航工具，二者互补，适用于不同的场景，核心区别如下：

特性	ui.navigate	ui.link
使用方式	程式调用（如按钮点击事件）	声明式创建（直接在页面中渲染链接）
动态性	支持条件跳转、参数动态拼接	仅支持静态路径和参数
历史记录	支持 <code>replace</code> 参数控制历史记录	仅添加新历史记录，无替换选项
过渡动画	依赖页面的 <code>transition</code> 配置	与页面的过渡动画一致
适用场景	动态导航（如登录后跳转、权限验证后跳转）	静态导航（如导航栏、页面底部链接）

最佳实践：

- 静态导航链接（如首页、关于页）使用 `ui.link`；
- 动态导航逻辑（如登录、筛选、分页）使用 `ui.navigate`。

八、总结

`ui.navigate` 是 NiceGUI 实现动态页面导航的核心工具，它封装了前端路由的底层逻辑，提供了简洁的 API 实现页面跳转、参数传递、历史记录管理等功能。其与 `ui.page` 的深度联动，让开发者可以轻松构建多页面应用；结合 `ui.session` 和装饰器，可实现权限验证、条件跳转等复杂的导航逻辑；通过过渡动画和刷新策略，还能优化用户的导航体验。

核心要点回顾：

- 基础用法：**通过 `to()`、`back()`、`forward()` 实现基础的页面跳转；
- 参数传递：**支持路径参数（资源标识）和查询参数（筛选条件）两种方式；
- 高级特性：**支持历史记录替换、强制刷新、过渡动画、动态条件跳转；
- 导航守卫：**通过装饰器和中间件实现权限拦截、登录检查；
- 最佳实践：**与 `ui.link` 配合，静态导航用 `ui.link`，动态导航用 `ui.navigate`。

掌握 `ui.navigate` 的使用，结合 `ui.page`、`ui.session`、`app.storage` 等核心工具，可完整构建出结构清晰、交互流畅的多页面 NiceGUI 应用，满足从简单页面切换到复杂权限控制的各类导航需求。